Loan Tracker

Documentation

---

1. Introduction

This document describes the implementation of a small application developed as part of a coding task. The purpose of the application is to track small loans given to colleagues, for example when someone forgets their lunch money.

In the described scenario, a colleague may borrow money during lunch. Over time, keeping track of these loans manually becomes difficult. The application provides a simple solution to record, view, and manage these loans.

The project was developed using **C#, .NET, and Visual Studio**, with a focus on good architecture, maintainability, and extensibility.

---

2. Problem Description

In many workplaces, colleagues occasionally lend money to each other for small expenses such as lunch. Without a structured system, it becomes difficult to remember:

- who borrowed money
- how much was borrowed
- when the loan was given

The goal of this application is to provide a simple system to record and manage these loans.

---

3. Application Features

The application offers the following functionality.

3.1 Person Management

Before recording loans, users can create and manage **persons (colleagues)**.

Each person contains the following information:

- Name
- Surname
- Email
- Address

This ensures that loans can always be associated with a specific person.

3.2 Unique Person Constraint

To avoid duplicate entries, the application enforces a **unique constraint on the combination of Name and Surname**.

This means that two persons cannot exist with the same name and surname combination.

Example:

Name	Surname	Allowed
Daniel	Müller	✅
Daniel	Schmidt	✅
Daniel	Müller	❌ duplicate

This constraint is implemented at the **database level using Entity Framework Core** with a composite unique index.

Example configuration:

```
modelBuilder.Entity<Person>()
    .HasIndex(p => new { p.Name, p.Surname })
    .IsUnique();
```

This guarantees **data consistency**.

3.3 Loan Creation

The application allows users to create a new loan.

Each loan contains:

- The person who borrowed the money
- The amount of money
- The date of the loan

The person must be selected from the list of existing persons, ensuring correct assignment of loans.

3.4 Loan Overview

All loans are displayed in a table view.

The table shows:

- Person
- Amount
- Date

The overview allows users to quickly see all recorded loans.

3.5 Loan Deletion

Users can delete loans from the list.

This is useful if:

- a loan was entered incorrectly
- a loan has already been repaid

3.6 Total Loan Amount

The application calculates and displays the **total amount of money currently lent**.

This gives users a quick overview of the total outstanding loans.

3.7 Sorting

The loan overview table allows sorting by columns.

Users can sort loans by:

- Person
- Amount
- Date

This improves usability and makes it easier to analyze loan data.

---

4. Architecture

The application uses a **layered architecture combined with the MVVM pattern** to ensure good maintainability and separation of concerns.

The solution consists of several projects.

4.1 Domain Layer

The Domain layer contains the core business entities and interfaces.

Example entities:

- Person
- Loan

The Domain layer represents the **business model** and contains no dependencies on external frameworks.

4.2 Service Layer

The Service layer contains the **business logic** of the application.

Responsibilities include:

- Creating loans
- Deleting loans
- Managing persons
- Validating data
- Applying business rules

This ensures that business logic is separated from the user interface.

4.3 Persistence Layer

The Persistence layer handles **data storage and retrieval**.

It uses **Entity Framework Core** for database operations.

Repositories are implemented to abstract database access from the rest of the application.

4.4 User Interface Layer

The user interface is implemented using **WPF (Windows Presentation Foundation)**.

The UI follows the **MVVM (Model-View-ViewModel)** pattern.

Responsibilities:

Component	Responsibility
View	UI layout and bindings
ViewModel	UI logic and commands
Model	Data entities

This pattern improves:

- testability
- maintainability
- separation of concerns

---

5. Technologies Used

The following technologies were used to develop the application:

- C#

- .NET
- WPF
- Entity Framework Core
- Material Design for WPF
- Dependency Injection
- NLog logging framework

These technologies provide a modern and extensible application architecture.

---

6. Software Design Principles

Several software engineering principles were applied during development.

Separation of Concerns

Each layer of the application has a clear responsibility.

Dependency Injection

Services and repositories are injected using dependency injection to reduce coupling.

SOLID Principles

The design follows important SOLID principles, including:

- Single Responsibility Principle
- Dependency Inversion Principle

These principles improve maintainability and extensibility.

---

7. Error Handling and Logging

The application includes logging to record important events and errors.

Examples of logged events:

- Creating a loan
- Deleting a loan
- Database errors

Logging helps with debugging and monitoring application behavior.

---

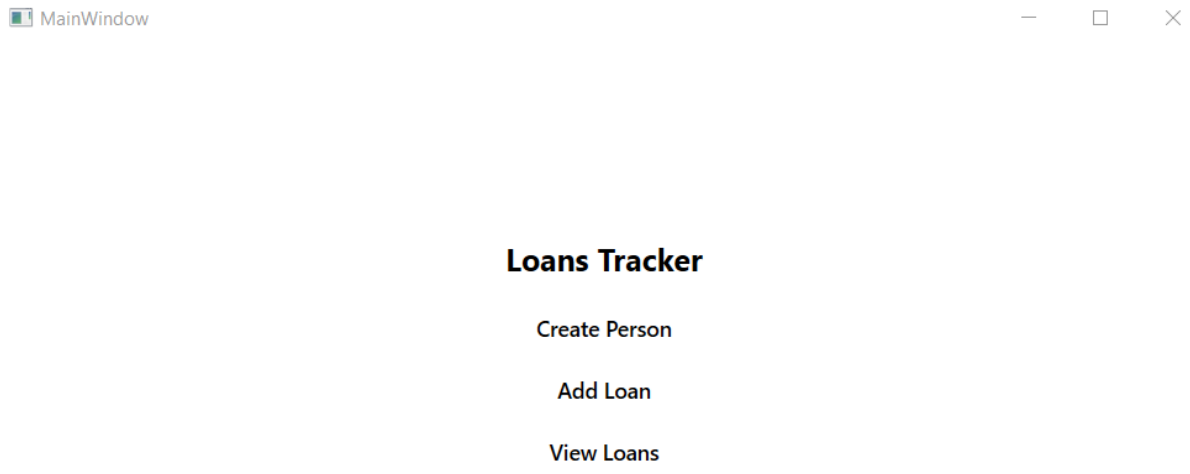
8. User Interface

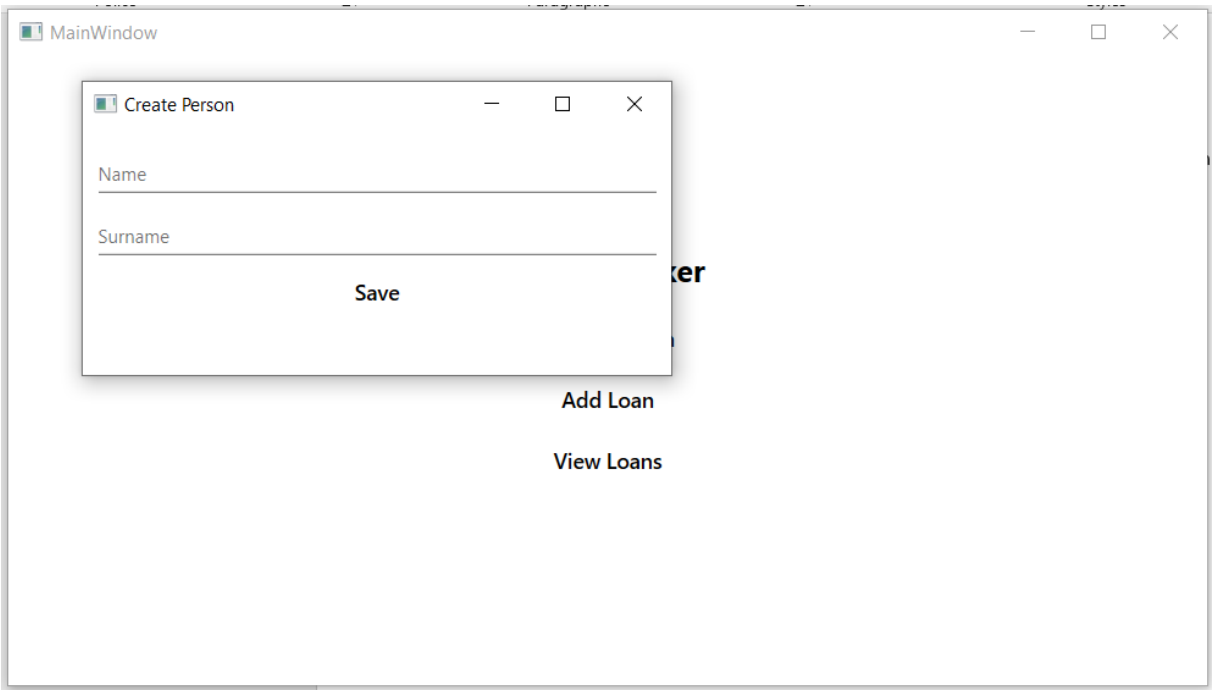
The application provides a simple and intuitive user interface.

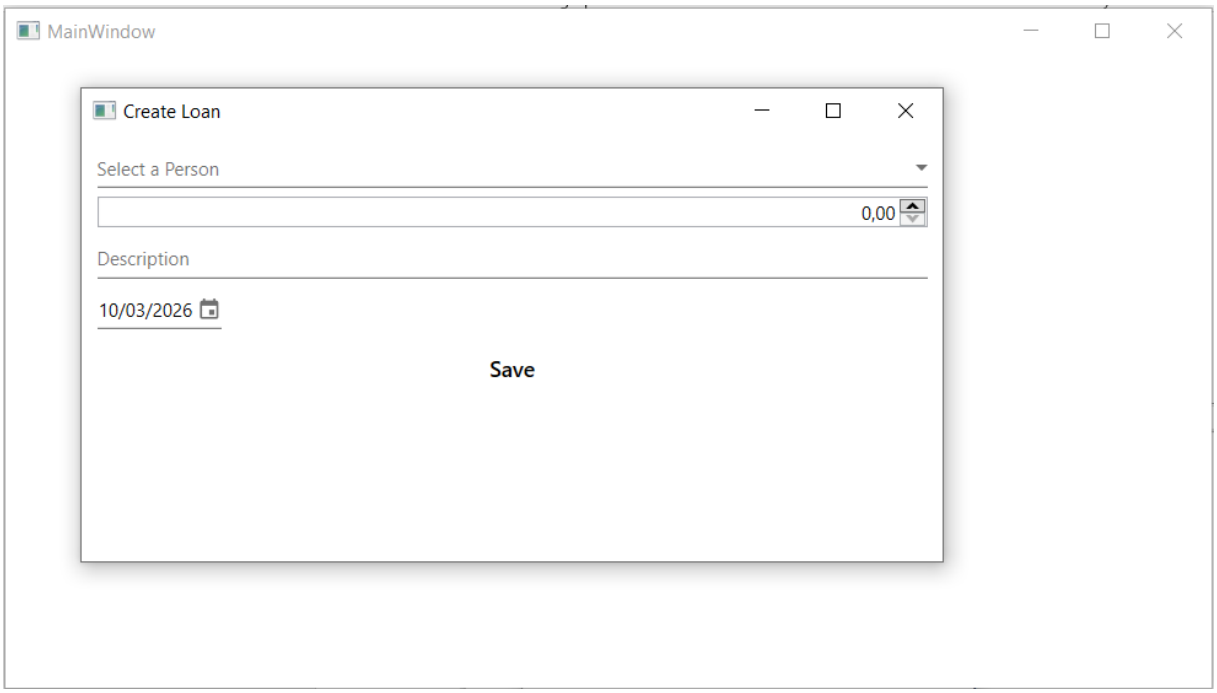
Main UI components include:

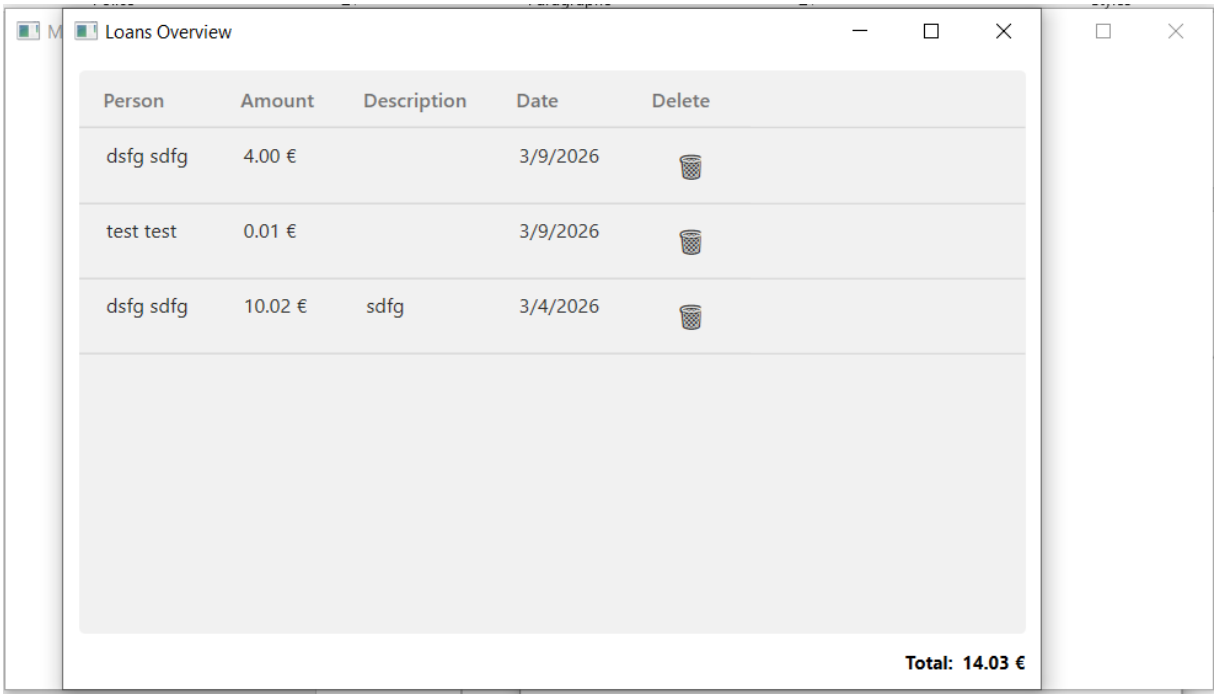
- Person creation form
- Loan creation form
- Loan overview table
- Delete buttons for removing loans
- Total loan amount display

The UI uses **Material Design components** to provide a modern and clean appearance.









9. Running the Application

To run the application:

1. Open the Visual Studio solution file (.sln).
2. Restore NuGet packages if required.
3. Build the solution.
4. Run Update-Database in Paket-Manager-Konsole
5. Run the application.

The application will start and allow users to create persons and record loans.

---

10. Possible Improvements

The application could be extended with additional features, such as:

- Editing existing loans
- Tracking repayments
- Filtering loans by person
- Exporting loan data
- Search functionality
- User authentication

These improvements would increase the usefulness of the system in real-world scenarios.

---

11. Conclusion

The project demonstrates a simple but well-structured implementation of a loan tracking system.

Even though the task is relatively small, the application was implemented using professional software development practices, including layered architecture, MVVM design pattern, dependency injection, and logging.

This ensures that the application is maintainable, extensible, and easy to understand.